Userstory - BlaBla Carpooling App

Overview

The BlaBla project is a **ride-sharing web application** built with Node.js, Express, MongoDB, and EJS, following the MVC architecture. Over the span of 7 days, it evolves into a complete carpooling platform. Users can create accounts, manage their profiles with personal details and profile pictures, and log in as either drivers or passengers. Drivers can publish rides, manage their listings, and confirm payments, while passengers can search, view, and book rides with flexible payment options and cancellation features. A key highlight is **map integration**, which visually displays routes between source and destination on ride details and booking pages. By the end of the plan, the app delivers a **realistic carpooling workflow** with authentication, role-based access, booking management, and interactive maps.

Day 1 – Project Setup & Authentication

- Setup project with **MVC structure** (controllers, routes, models, middlewares, views, public).
- Initialize Node.js project with dependencies.
- Setup MongoDB connection.
- Create a **User model** with fields.
- Implement **Signup**: register new users with hashed password.
- Implement **Login**: authenticate and return JWT token.
- Middleware: request logger, auth check, error handler.
- Build simple frontend pages:
    - **Home**: guest view (welcome + links).
    - **Signup/Login** forms.
    - **Profile Page** (placeholder for now).

Day 2 – Profile Management

- Extend **Profile Page** to show user info after login.
- Add ability to **update profile details**.
- Add profile picture uploading (store image locally in server).
- Create routes/controllers for updating profile fields.
- Frontend:
    - **Profile View** → shows all user details + profile picture.
    - **Profile Edit Form** → allows updates and picture upload.
- Ensure only logged-in user can view and edit their own profile.

Day 3 – Ride Publishing

- Create a **Ride model** with fields.
- Logged-in **drivers** can publish rides.
- Add **Publish Ride Page** (form).
- Store rides in MongoDB linked to driver.
- Extend **Profile** → **My Rides** section for drivers (shows rides they published).
- Restrict publish access → only logged-in drivers.

Day 4 – Ride Listing & Searching

- Create **Rides Listing Page** showing all published rides (accessible to everyone).
- Create a **Search Page** with filters like : departure time, price, date, no. of seats.

- Show results list with driver name, time, price, seats.
- Add **Ride Details Page**: clicking a ride shows full details (driver info, profile picture, all info of ride).
- Add map placeholder section (to be integrated later).

Day 5 – Ride Booking

- Create **Booking model** with fields: rideId, customerId, seatsBooked, status, paymentStatus.
- Logged-in passengers can **book rides** from search results or ride details page.
- Booking rules:
    - Cannot book own ride.
    - Seats booked ≤ available seats.
- After booking → reduce available seats.
- Extend **Profile** → **My Bookings** section: shows rides booked as passenger.

Day 6 – Ride Management, Cancellations & Payments

- Drivers can manage rides: edit or delete their rides.
- **Cancellations**:
    - Passengers can cancel bookings (before the ride starts).
    - Drivers can cancel rides (cancels all related bookings).
- **Payments**:
    - Option for Cash or Online during booking.
    - Online → Opens API page → Paid/Failed.

    - Cash → Pending until driver confirms.

- Profile Enhancements:
    - **My Rides** → drivers can cancel or confirm cash payments.
    - **My Bookings** → passengers see status (confirmed, cancelled, completed) + payment state.

Day 7 – Map Integration

- Integrate **Map with API** for route display.
- On **Ride Search Results Page** and **Ride Details Page**: show route map (source → destination).
- On **Profile** → **My Bookings**: booked rides also show the same map.
- On **Profile** → **My Rides**: drivers see map for their rides.
- Store coordinates for departure/destination in ride documents.
- Ensure maps are interactive and render dynamically per ride.